MANUAL TÉCNICO DEL PROGRAMADOR

Proyecto: Juego de Naves en Consola — Actividad 8
Institución: E.E.S.T. N° 1 “Escuela de Educación Técnica N.° 1 "Eduardo Ader" ” – Vicente López
Asignatura: Laboratorio de Programación (LPR) – 5° Año
Autor: Mikaela Batirola
Fecha: 21 de octubre de 2026 | **Versión:** v1.0.0

CONTROL DE VERSIONES Y CHANGELOG

Versión	Fecha	Autor	Descripción de Cambios
v1.0.0	21/10/2026	Mikaela Batirola	Versión inicial del manual técnico de juego de naves en C++.

1. ARQUITECTURA DEL SISTEMA Y ENTORNO DE DESARROLLO

Requisitos del entorno

- **Lenguaje:** C++
- **Compilador:** g++ compatible con Windows
- **IDE:** Visual Studio Code
- **Sistema operativo:** Windows
- **Terminal:** PowerShell

Estructura de directorios del repositorio

```
juegodenaves/
├── .gitignore
├── LICENSE
├── README.md
├── docs/
│   ├── InformeEEST1_LPR2026_ACT08_ApellidoNombre_v1.0.0.pdf
│   ├── CHANGELOG.md
│   └── manuales/
│       ├── manual_programador_v1.0.0.pdf
│       └── manual_usuario_v1.0.0.pdf
├── src/
│   └── main.cpp
└── capturas/
    └── combate_consola.png
```

El archivo `src/main.cpp` contiene el código fuente principal. Los ejecutables generados durante la compilación, como `juegodenaves.exe`, no se incluyen en el repositorio porque deben excluirse mediante el archivo `.gitignore`.

La carpeta `docs/` contiene la documentación formal del proyecto, mientras que `capturas/` almacena las evidencias de ejecución.

2. DOCUMENTACIÓN DE CLASES Y MÉTODOS (`src/main.cpp`)

Módulo 1: Función `void gotoxy(int x, int y)`

- **Propósito:** Posicionar el cursor en una coordenada determinada de la consola.
- **Firma:** `void gotoxy(int x, int y)`
- **Funcionamiento:** Obtiene el identificador de la salida de la consola y utiliza `SetConsoleCursorPosition()` para cambiar la posición del cursor.
- **Parámetros:** x representa la columna e y representa la fila.
- **Uso:** Permite dibujar la nave, los asteroides y las balas en diferentes posiciones sin tener que limpiar toda la pantalla en cada actualización.

Módulo 2: Función `void ocultarCursor()`

- **Propósito:** Ocultar el cursor de la consola durante la partida.
- **Firma:** `void ocultarCursor()`
- **Funcionamiento:** Utiliza `GetConsoleCursorInfo()` indirectamente mediante la estructura `CONSOLE_CURSOR_INFO` y `SetConsoleCursorInfo()` para establecer la visibilidad del cursor.
- **Uso:** Mejora la presentación visual del juego mientras los objetos se desplazan.

Módulo 3: Clase `Nave`

La clase `Nave` representa al jugador y contiene sus datos y las acciones que puede realizar durante la partida.

Atributos privados (`private`):

- `int x`: almacena la posición horizontal de la nave.
- `int y`: almacena la posición vertical de la nave.
- `int vidas`: almacena la cantidad de vidas disponibles.

Estos atributos no pueden modificarse directamente desde `main()`, ya que están declarados como privados. Se accede a la información necesaria mediante métodos públicos.

Métodos públicos (`public`):

- **`Nave(int _x, int _y, int _vidas)`:** constructor que inicializa la posición y las vidas del jugador.
- **`getX()`:** devuelve la posición horizontal.
- **`getY()`:** devuelve la posición vertical.
- **`getVidas()`:** devuelve la cantidad de vidas.
- **`reducirVida()`:** disminuye en uno la cantidad de vidas.
- **`pintar()`:** dibuja la nave en la consola.
- **`borrar()`:** elimina visualmente la nave de su posición anterior.
- **`mover(char tecla)`:** actualiza la posición según la tecla presionada y los límites establecidos.

Funcionamiento: La clase reúne los atributos y métodos relacionados con el jugador. De esta manera, el movimiento, la representación visual y el control de vidas se organizan dentro de un mismo objeto.

Encapsulamiento: Los atributos `x`, `y` y `vidas` son privados. Sin embargo, en el código base, el método `reducirVida()` no comprueba si las vidas ya llegaron a cero. Por eso, el control de las colisiones en `main()` debe evitar que se sigan reduciendo innecesariamente.

Módulo 4: Clase `Asteroide`

La clase `Asteroide` representa los obstáculos que aparecen durante el juego.

Atributos privados:

- `int x`: posición horizontal del asteroide.
- `int y`: posición vertical del asteroide.

Métodos públicos:

- **`Asteroide(int _x, int _y)`:** inicializa la posición del asteroide.
- **`getX()`:** devuelve su posición horizontal.
- **`getY()`:** devuelve su posición vertical.
- **`pintar()`:** muestra el asteroide utilizando el carácter `0`.
- **`borrar()`:** limpia su posición anterior.
- **`mover()`:** desplaza el asteroide hacia abajo y, al superar el límite inferior, lo vuelve a ubicar en la parte superior con una posición horizontal aleatoria.

Funcionamiento: Cada objeto `Asteroide` mantiene su propia posición. Esto permite que varios asteroides se desplacen de manera independiente durante la partida.

Módulo 5: Clase `Bala`

La clase `Bala` representa los proyectiles que dispara el jugador.

Atributos privados:

- `int x`: posición horizontal de la bala.
- `int y`: posición vertical de la bala.

Métodos públicos:

- **`Bala(int _x, int _y)`:** inicializa la posición del proyectil.
- **`getX()`:** devuelve su posición horizontal.
- **`getY()`:** devuelve su posición vertical.
- **`mover()`:** borra la posición anterior, desplaza la bala hacia arriba y la dibuja nuevamente.
- **`fueraDeRango()`:** devuelve `true` cuando la bala supera el límite superior del área de juego.

Funcionamiento: Cuando el jugador presiona la barra espaciadora, se crea un nuevo objeto `Bala`. El programa actualiza su posición hasta que sale del área de juego o alcanza un asteroide.

Módulo 6: Función principal `int main()`

La función `main()` coordina el funcionamiento general del juego y conecta los objetos creados a partir de las clases.

Inicialización:

- `srand(time(NULL))` configura la semilla utilizada para generar posiciones aleatorias.
- `getline(cin, nombrePiloto)` permite ingresar el nombre completo del jugador.
- `system("cls")` limpia la consola antes de iniciar la partida.
- `ocultarCursor()` oculta el cursor.

Creación de objetos:

- `Nave jugador(35, 20, 3)` crea la nave en la posición inicial con tres vidas.
- `vector<Asteroide*> asteroides` almacena punteros a los asteroides.
- `vector<Bala*> balas` almacena punteros a las balas disparadas.
- Un ciclo `for` crea cinco asteroides iniciales.

Bucle principal:

Mientras `gameOver` sea `false`, el programa actualiza la información del jugador, lee las teclas, mueve las balas y los asteroides y comprueba las colisiones.

Control del teclado:

- `_kbhit()` comprueba si hay una tecla disponible.
- `_getch()` lee la tecla sin necesidad de presionar Enter.
- `tolower()` convierte las letras a minúsculas.
- La barra espaciadora crea una nueva bala y las teclas W, A, S y D controlan el movimiento.

Detección de colisiones:

El programa compara las coordenadas de los asteroides con las de la nave para detectar impactos. También compara las posiciones de las balas con las de los asteroides para determinar si un disparo alcanza un obstáculo.

Finalización:

Cuando las vidas del jugador llegan a cero, la variable `gameOver` pasa a `true`. El bucle termina y la consola muestra el mensaje de finalización de la partida.

Gestión de memoria: El código base utiliza `new` para crear asteroides y balas dinámicamente, y `delete` para liberar algunos objetos cuando dejan de utilizarse. Para una versión más robusta, se recomienda liberar también los objetos que queden almacenados

en los vectores al finalizar la partida y evitar reemplazar objetos sin liberar previamente la memoria que ocupan.

3. INSTRUCCIONES DE COMPILACIÓN Y EJECUCIÓN

Compilación desde PowerShell

Ubicándose en la carpeta raíz del proyecto, ejecutar:

g++ src/main.cpp -o src/juegodenaves.exe

Si la compilación termina sin errores, se generará el archivo ejecutable.

Para iniciar el programa, ejecutar:

.\src\juegodenaves.exe

El juego solicitará el nombre y apellido del piloto y luego iniciará la partida en la consola.

Diagnóstico de errores frecuentes

Error: `'g++' no se reconoce como nombre de un cmdlet`.

- **Causa:** El compilador no está instalado o su ubicación no está incluida en la variable de entorno PATH.
- **Solución:** Comprobar la instalación de MinGW-w64 y la configuración del PATH.

Error relacionado con `windows.h` o `conio.h`.

- **Causa:** El compilador o el sistema utilizado no admite las bibliotecas específicas de Windows.
- **Solución:** Utilizar un entorno de compilación compatible con Windows.

El ejecutable no aparece.

- **Causa:** La compilación no terminó correctamente o se ejecutó el comando desde una carpeta incorrecta.
- **Solución:** Revisar los mensajes de error y comprobar la ruta `src/juegodenaves.exe`.

Los asteroides o las balas dejan caracteres en la consola.

- **Causa:** El dibujo de los objetos depende de las coordenadas y del borrado de sus posiciones anteriores.
- **Solución:** Verificar los métodos `pintar()`, `borrar()` y `mover()`, y revisar que las coordenadas se actualicen correctamente.

4. REFERENCIAS

- **cplusplus.com** — Clases y Objetos en C++: https://cplusplus.com/doc/tutorial/classes/
- **GeeksforGeeks** — Modificadores de Acceso en C++: https://www.geeksforgeeks.org/access-modifiers-in-c/
- **Microsoft Learn** — Constructores e Inicialización de Clases: https://learn.microsoft.com/es-es/cpp/cpp/constructors-cpp
- **Git Documentation** — Flujo básico y manejo de repositorios: https://git-scm.com/doc